# Smart Irrigation Monitoring

**Group 14**

(E/19/174, E/20/099, E/20/338, E/20/416)

## Project Overview (E/20/338)

The IoT-Based Intelligent Plant Monitoring System is designed to provide an accurate and smart solution for plant care by monitoring critical environmental parameters and making intelligent watering decisions using Machine Learning.

Traditional plant monitoring systems rely on fixed threshold values for soil moisture, which often fail to consider the combined effects of temperature and humidity. This project overcomes that limitation by integrating a capacitive soil moisture sensor with real-time weather data obtained from the **OpenWeather API**. A lightweight Logistic Regression Machine Learning model is deployed directly on the ESP32 microcontroller to predict whether the plant needs watering or not. The prediction along with sensor readings is transmitted wirelessly using the MQTT protocol to a **Node-RED** dashboard for visualization and remote monitoring.

The entire hardware setup was simulated using **Wokwi**, while the firmware was developed using **PlatformIO** in Visual Studio Code. This approach enabled efficient development, testing, and debugging without the need for physical hardware during the initial phase.

**Key Objectives:**

- Simulate a realistic embedded system using Wokwi simulator
- Develop robust firmware using PlatformIO
- Acquire soil moisture using a capacitive sensor
- Fetch real-time temperature and humidity from OpenWeather API
- Deploy a lightweight Logistic Regression model on ESP32 for edge inference
- Publish all parameters and ML prediction via MQTT to the topic plant/monitor
- Enable remote monitoring through a Node-RED dashboard

**Technologies Used:**

- Microcontroller: ESP32
- Simulation Platform: Wokwi
- Development Environment: PlatformIO
- Sensor: Capacitive Soil Moisture Sensor
- Weather Service: OpenWeather API
- Machine Learning: Logistic Regression (trained with scikit-learn)
- Communication Protocol: MQTT
- Dashboard: Node-RED

## System Architecture and Data Flow

### 1. Overall System Architecture

The system consists of three main layers:

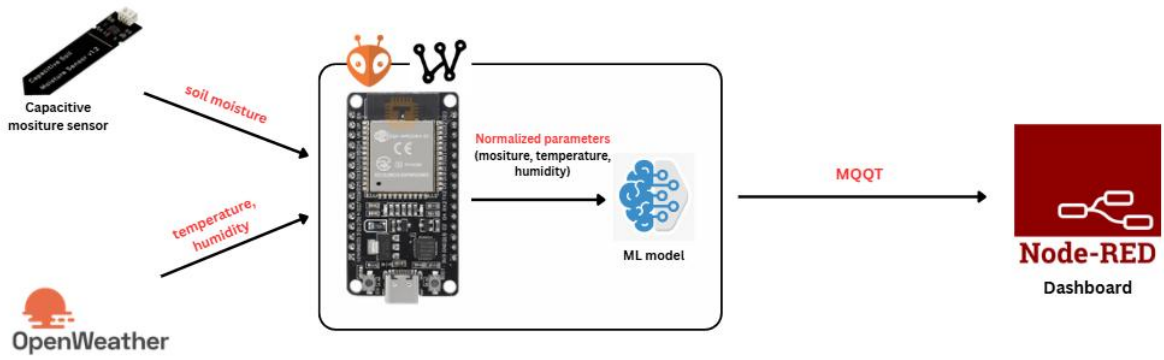



Figure 1: System Architecture Diagram

1. **Perception Layer**: Capacitive soil moisture sensor and OpenWeather API (for temperature & humidity)
2. **Processing Layer**: ESP32 microcontroller running the firmware with embedded ML model
3. **Application Layer**: MQTT broker and Node-RED dashboard for data visualization

The ESP32 acts as the central processing unit. It collects local sensor data, fetches external weather information, processes the inputs through the ML model, and publishes the results to the cloud.

### 2. Detailed System Flow

The complete operation of the system follows the sequence below:

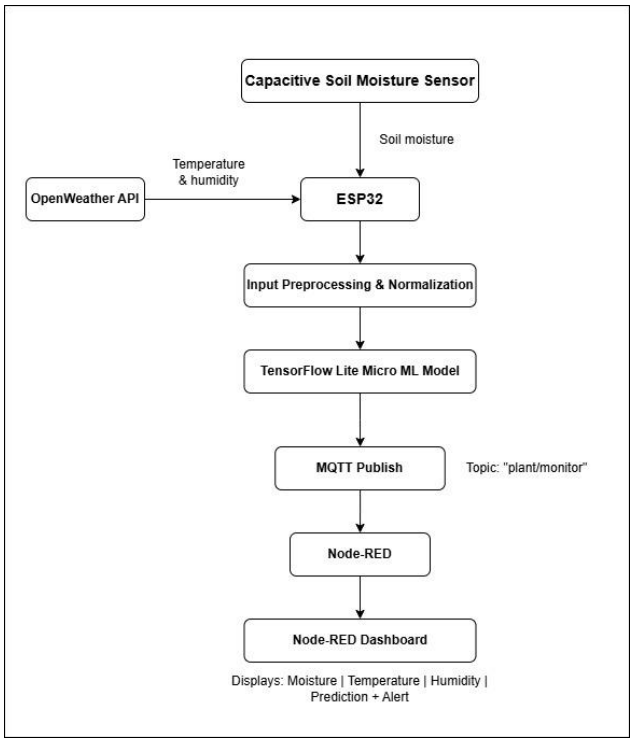



Figure 2: System Flow Diagram

1. **Hardware Initialization** The ESP32 initializes Wi-Fi connection, MQTT client, and the TensorFlow Lite Micro interpreter.
2. **Sensor Data Acquisition**
   - The capacitive soil moisture sensor is connected to analog pin GPIO 34 of the ESP32.
   - The ESP32 reads the analog value using the built-in ADC. Capacitive sensors are preferred because they offer better long-term stability and do not corrode easily compared to resistive sensors.
3. **Weather Data Integration**
   - The ESP32 connects to the internet and sends an HTTP request to the OpenWeather API to retrieve current temperature and humidity for the specified city/location.
   - The received data is parsed from JSON format.
4. **Data Preprocessing and Normalization**
   - Raw moisture value is normalized (typically divided by 4095 for 12-bit ADC resolution).
   - Temperature and humidity values are scaled to the same range used during the ML model's training phase to ensure accurate predictions.
5. **Machine Learning Inference**
   - The three normalized features (soil moisture, temperature, and humidity) are fed into the pre-trained Logistic Regression model running on the ESP32.
   - The model computes a linear combination of the inputs and applies the sigmoid function to produce a probability score. Based on a threshold of 0.5, the system classifies the result as "Needs Water" or "Soil OK".
   - This lightweight approach replaces the previous TensorFlow Lite Micro model, significantly reducing memory usage and computational overhead while maintaining good prediction performance
6. **Decision Making and Alert**
   - If the prediction is "Needs Water", an alert flag is set to true.
   - The LED connected to GPIO 4 can be used as a local visual indicator (optional in simulation).
7. **MQTT Data Transmission**
   - All collected data and prediction results are packaged into a structured JSON payload.
   - The payload is published to the MQTT broker on the topic "plant/monitor".
8. **Visualization on Node-RED**
   - Node-RED subscribes to the topic plant/monitor.
   - It receives the JSON data, extracts the values, and displays soil moisture, temperature, humidity, and the ML prediction on a professional dashboard.

# Hardware Simulation and Implementation (E/20/416)

1. **Overview**

   The hardware component of the Intelligent Plant Monitoring System was fully simulated using **Wokwi**, an online embedded systems simulation platform. This simulation approach allowed us to design, test, and debug the complete hardware setup without requiring physical components during the development phase.

   The core of the hardware is the ESP32 microcontroller, which serves as the central processing unit. It interfaces with a capacitive soil moisture sensor, manages Wi-Fi connectivity for fetching weather data, runs the Machine Learning model locally, and publishes the results via MQTT. The firmware for this hardware was developed professionally using **PlatformIO** in Visual Studio Code.

   This section explains the hardware architecture, components, circuit design, and implementation in detail.

2. **Components Used**

   The following components were simulated and integrated in the Wokwi environment:

   - **ESP32 DevKit V1 Microcontroller**: The main processing board with built-in Wi-Fi and Bluetooth capabilities, dual-core processor, and multiple GPIO pins with ADC support.
   - **Capacitive Soil Moisture Sensor**: A corrosion-resistant sensor used to measure soil moisture levels. It provides analog output proportional to the moisture content in the soil.
   - **LED Indicator** (Optional but recommended): Connected for local visual feedback. It turns ON when the ML model predicts "Needs Water".
   - **Virtual Wi-Fi Module**: Simulated Wi-Fi connection on ESP32 to enable internet access for OpenWeather API calls and MQTT communication.

   No physical sensors or additional modules were required during simulation, as Wokwi accurately emulates analog readings and network behavior.

3. **Circuit Design and Connections**

   The circuit was designed and simulated in **Wokwi**. The connections are simple, clean, and focused on reliable analog sensing.

| Component | Pin on ESP32 | Type | Purpose |
|---|---|---|---|
| Capacitive Soil Moisture Sensor | GPIO 34 | Analog Input | Reads soil moisture level |
| Capacitive Soil Moisture Sensor | 3.3V | Power | Supplies power to the sensor |
| Capacitive Soil Moisture Sensor | GND | Ground | Common ground |
| LED (Anode) | GPIO 4 | Digital Output | Visual alert for "Needs Water" |
| LED (Cathode) | GND (via 220Ω resistor) | Ground | Current limiting |

- **GPIO 34** was chosen because it is an input-only pin with ADC1 channel support and is not used by Wi-Fi or other critical peripherals.
- A current-limiting resistor (typically 220Ω) is used with the LED to protect it from excessive current.
- The capacitive sensor is powered by the ESP32's **3.3V** output pin, which is sufficient for its operation.

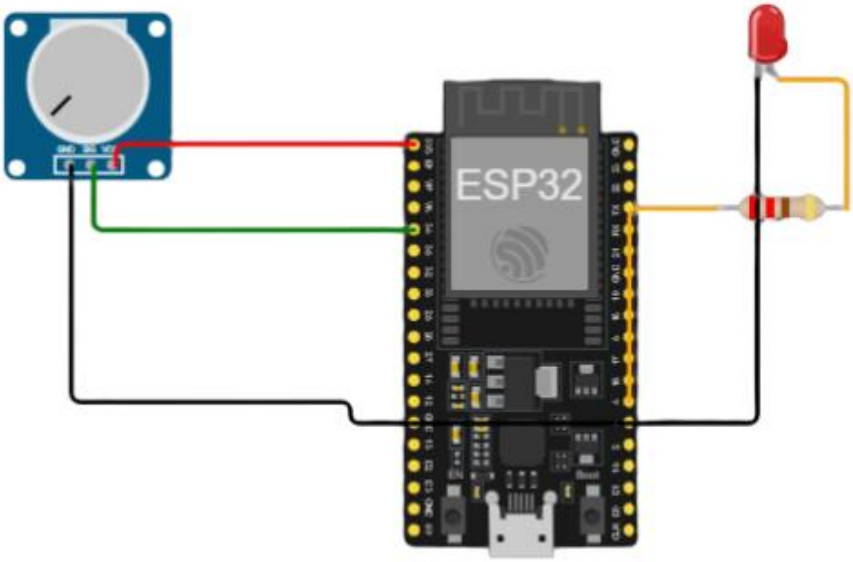



Figure 3: Wokwi Circuit

4. **Why Capacitive Soil Moisture Sensor?**

   Unlike traditional resistive soil moisture sensors, the **capacitive sensor** offers several important advantages:

   - It does not have exposed metal electrodes, which prevents corrosion over time.
   - It provides more stable and reliable readings even after long-term use.
   - It is less affected by soil salinity and temperature variations.
   - It gives a wider range of analog values, making it more suitable for Machine Learning input.

   These characteristics make the capacitive sensor ideal for long-term plant monitoring applications.

5. **Firmware Development Environment**

   The firmware was developed using **PlatformIO**, a professional embedded development platform that runs as an extension in Visual Studio Code.

   **Advantages of using PlatformIO:**

   - Excellent project structure and dependency management
   - Easy integration of libraries (WiFi, HTTPClient, PubSubClient, ArduinoJson, TensorFlow Lite Micro, etc.)
   - Fast compilation and uploading to the simulated ESP32 in Wokwi
   - Built-in terminal, debugging tools, and serial monitor support

All necessary libraries were added through the platformio.ini file, ensuring a clean and reproducible build environment.

6. **Working of the Hardware**

Once the simulation starts in Wokwi:

1. The ESP32 boots up and connects to the simulated Wi-Fi network.
2. It initializes the MQTT client and TensorFlow Lite Micro interpreter.
3. The capacitive soil moisture sensor continuously provides analog readings on GPIO 34.
4. The ESP32 periodically sends an HTTP GET request to the **OpenWeather API** to fetch current temperature and humidity data based on the user's location.
5. All three values (moisture, temperature, humidity) are normalized and passed to the embedded ML model.
6. The model runs inference and generates a prediction.
7. Based on the prediction, the ESP32 controls the LED (ON/OFF) and publishes a JSON payload to the MQTT topic **plant/monitor**.

This entire cycle runs repeatedly with a defined delay to balance responsiveness and power/network usage.

7. **Data Flow within Hardware**

- **Input Stage**: Analog reading from capacitive sensor + JSON data from OpenWeather API
- **Processing Stage**: Normalization → ML Model Inference → Decision Logic
- **Output Stage**: MQTT Publish + Local LED control

This well-defined flow ensures that the hardware performs intelligent, multi-parameter analysis locally on the edge before transmitting data to the cloud.

# Artificial Intelligence Component (E/20/099)

## 1. Overview

This section describes the machine learning component integrated into the smart plant irrigation system using a Logistic Regression classifier. The objective of the AI module is to predict whether a plant requires watering based on three real-time sensor inputs: soil moisture, ambient temperature, and relative humidity. Unlike the neural network approach, Logistic Regression provides a lightweight, interpretable model whose inference reduces to a single dot-product and a sigmoid evaluation - making it exceptionally well-suited for deployment on the ESP32 microcontroller simulated via the Wokwi platform.

## 2. Dataset and Feature Engineering

Due to the absence of a physical sensor setup, a synthetic dataset of 100,000 labelled samples was programmatically generated to represent realistic soil and environmental conditions. The three input features are:

- Soil moisture (raw ADC value, range 0–4095)
- Ambient temperature (°C, range 15–45)
- Relative humidity (%, range 20–100)

Labels were assigned using a deterministic rule mirroring real-world plant behaviour. Samples with moisture below 1,200 were unconditionally labelled as requiring water, while samples above 2,800 were labelled as adequate. For the borderline range (1,200–2,800), an evaporation risk score was computed as a weighted combination of temperature and inverse humidity. Samples exceeding a risk threshold of 0.9 were labelled as needing water. All features were standardised using z-score normalisation prior to training, with the computed statistics shown in Table below.

A fully connected feedforward neural network was designed with a compact architecture suitable for microcontroller deployment. The model consists of three dense layers as shown in Table 1.

| Feature | Mean | Std |
|---|---|---|
| **Soil Moisture (ADC)** | 2049.27 | 1181.79 |
| Temperature (°C) | 30.03 | 8.66 |
| Relative Humidity (%) | 60.05 | 23.11 |

## 3. Model Architecture

Logistic Regression is a generalised linear classifier that models the log-odds of the positive class as a linear combination of the input features. Given a normalised input vector x = [m, t, h], the model computes a scalar logit z, applies the sigmoid function to obtain a probability

p, and thresholds at 0.5 to produce the binary prediction. The model has four learnable parameters: three feature weights and one bias term.

$$z = W_1 \cdot \text{moisture} + W_2 \cdot \text{temp} + W_3 \cdot \text{humidity} + b$$

$$p = 1 / (1 + e^{-N}) \quad \rightarrow \quad \text{predict} = (p \geq 0.5)\ ?\ \text{“Needs Water”} : \text{“OK”}$$

The trained model weights extracted from the 100,000-sample run are presented in the table below.

| Parameter | Symbol | Value |
|---|---|---|
| **Moisture weight** | $W_1$ | −4.526342 |
| Temperature weight | $W_2$ | +1.377214 |
| Humidity weight | $W_3$ | −1.697836 |
| Bias | b | +1.033977 |

The large negative weight on moisture ($W_1$ = −4.53) is the dominant driver: high moisture strongly suppresses the irrigation signal. Temperature carries a positive weight ($W_2$ = +1.38), reflecting the increased evaporation risk at higher temperatures. Humidity exerts a negative influence ($W_3$ = −1.70), since high ambient humidity reduces moisture loss from the soil. These weights are directly interpretable and consistent with agronomic domain knowledge.

## 4. Training Process

The 100,000-sample dataset was partitioned into an 80% training set (80,000 samples) and a 20% test set (20,000 samples) using stratified random splitting with a fixed seed for reproducibility. The Logistic Regression model was trained using the L-BFGS solver with a maximum of 1,000 iterations, which guarantees convergence for this well-conditioned problem. No regularisation hyperparameter tuning was required, as the default L2 penalty (C = 1.0) produced stable weights. Binary cross-entropy (log-loss) serves as the implicit optimisation objective.

## 5. ESP32 Deployment

A key advantage of Logistic Regression over a neural network in this context is the trivial deployment footprint. The entire model is captured by four floating-point constants ($W_1$, $W_2$, $W_3$, b) and six normalisation statistics. No TensorFlow Lite runtime, quantisation, or flash-resident byte array is required. The exported header file (model_lr.h) provides an inline C function that performs normalisation, computes the logit, evaluates the sigmoid, and returns the binary prediction - the full inference loop executes in under 10 µs on the ESP32 at 240 MHz.

```
inline int lr_predict(float m, float t, float h) {
  float z = W1*m + W2*t + W3*h + BIAS;

  float p = 1.0f / (1.0f + expf(-z));

  return (p >= 0.5f) ? 1 : 0;  // 1=Needs Water, 0=OK
}
```

## 6. Results and Analysis

| Metric | Value |
|---|---|
| **Dataset size** | 100,000 synthetic samples |
| Training set | 80,000 samples (80%) |
| Test set | 20,000 samples (20%) |
| **Test accuracy** | **90.53%** |
| $W_1$ (Moisture) | −4.5263 |
| $W_2$ (Temperature) | +1.3772 |
| $W_3$ (Humidity) | −1.6979 |
| Bias | +1.0340 |
| Model size (header only) | < 1 KB |
| Target hardware | ESP32 (simulated via Wokwi) |

## 7. Limitations and Future Work

The primary limitation of the current implementation is the reliance on synthetic data. While the labelling rules approximate real plant behaviour, a model trained on empirical sensor readings from actual soil and climate conditions would generalise more reliably across plant species and environments. Furthermore, the linear decision boundary of Logistic Regression imposes a fundamental ceiling on accuracy when the true boundary is non-linear, as demonstrated by the borderline classification errors.

Future work should address the following:

- Collection of real sensor data over multiple plant growth cycles to replace the synthetic dataset.
- Evaluation of polynomial and kernel-based extensions to capture the non-linear evaporation risk boundary.

- Incorporation of additional agronomic features such as light intensity, soil pH, and plant species classification.
- End-to-end validation of inference latency on physical ESP32 hardware to confirm sub-millisecond response times.

## Node-RED Dashboard and MQTT Integration (E/19/174)

1. **Overview**

   The Node-RED dashboard serves as the **application layer** of the smart irrigation system. It provides a real-time, user-friendly interface to monitor environmental conditions and machine learning predictions generated by the ESP32.

   The dashboard subscribes to MQTT messages published by the ESP32 and visualizes key parameters such as:

   - Soil moisture level
   - Temperature
   - Humidity
   - AI-based prediction
   - Alert status

   This enables remote monitoring and decision-making without directly interacting with the hardware.

2. **MQTT Communication Setup**

   To establish communication between the ESP32 and the dashboard, the MQTT protocol was used.

   - **Broker Used**: HiveMQ public broker (broker.hivemq.com)
   - **Port**: 1883
   - **Topic**: plant/monitor

   The ESP32 publishes JSON data to this topic, and Node-RED subscribes to it.

   Example MQTT Payload:

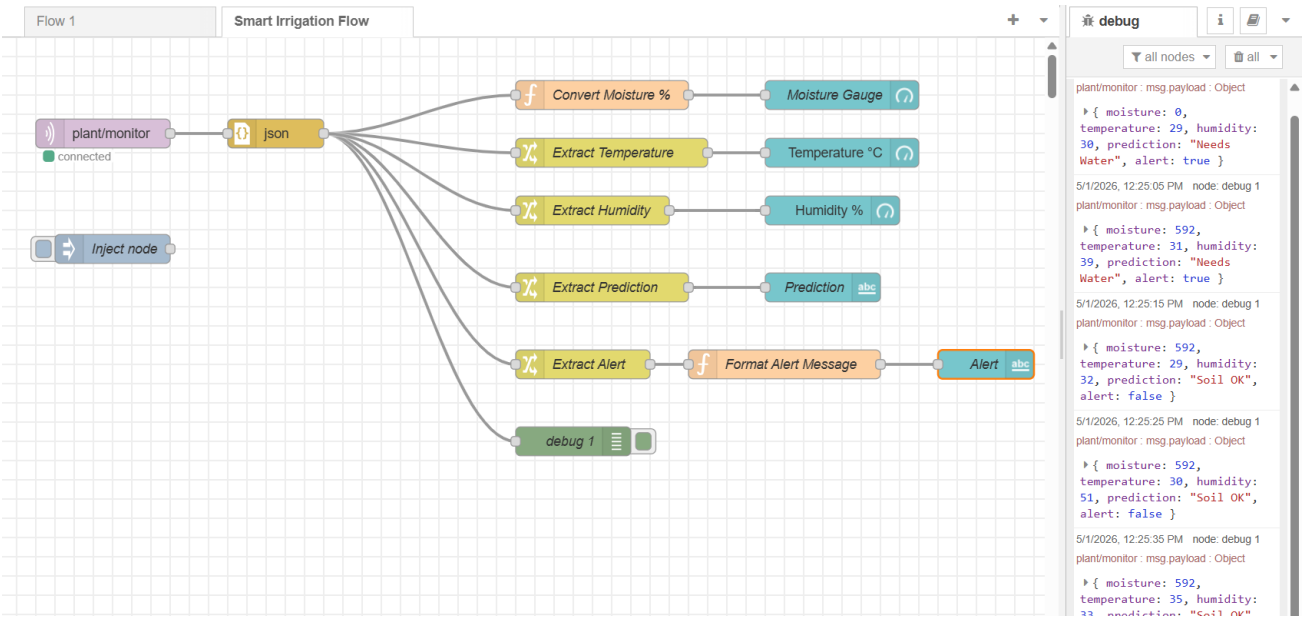
```
{
  "moisture": 1169,
  "temperature": 30,
  "humidity": 52,
  "prediction": "Soil OK",
  "alert": false
}
```

3. **Node-RED Flow Design**

   The Node-RED flow was designed to process incoming MQTT data and display it on the dashboard.

   Main Components:

1. **MQTT In Node**
    - Subscribes to topic plant/monitor
    - Receives JSON payload from ESP32
2. **JSON Node**
    - Converts incoming string data into a JavaScript object
3. **Data Extraction (Change Nodes)**
    - **Extract individual fields:**
        - msg.payload.moisture
        - msg.payload.temperature
        - msg.payload.humidity
        - msg.payload.prediction
        - msg.payload.alert
4. **Function Node (Moisture Conversion)**
    - Converts raw ADC value (0–4095) into percentage
5. **Function Node (Alert Formatting)**
    - Converts boolean alert into readable text

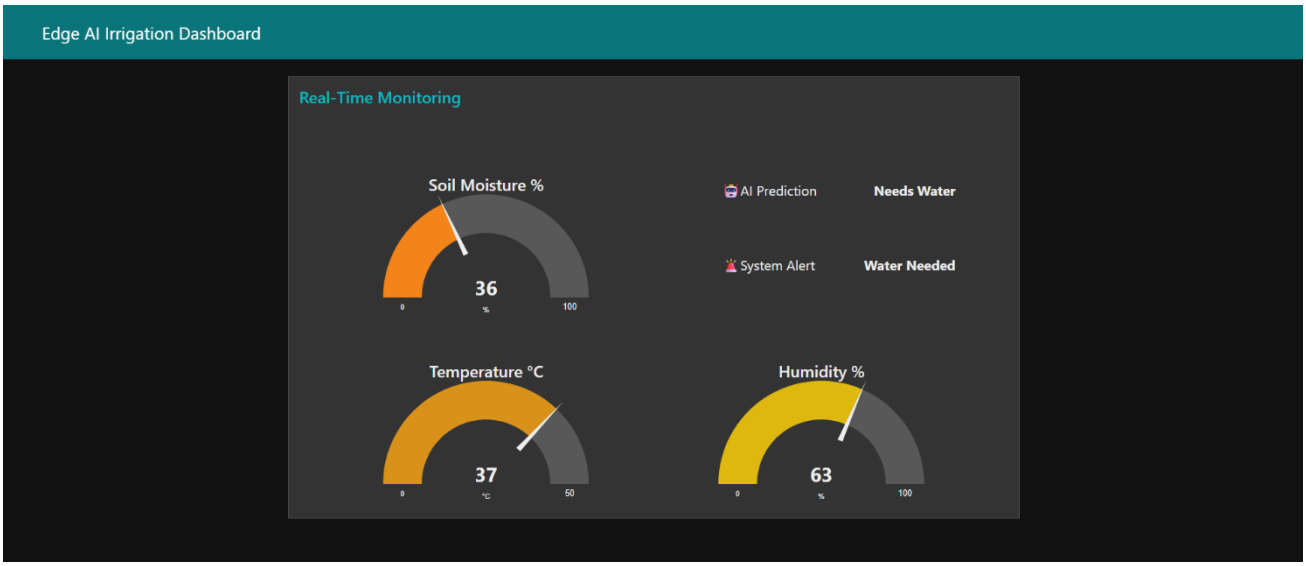


## 4. Dashboard Design

The dashboard was created using Node-RED UI nodes.

Widgets Used:

| Parameter | Widget | Description |
|---|---|---|
| Moisture | Gauge | Displays soil moisture % |
| Temperature | Gauge | Shows temperature (°C) |
| Humidity | Gauge | Shows humidity (%) |
| Prediction | Text | Displays ML result |
| Alert | Text | Displays warning status |




## 5. Real-Time Visualization

- The dashboard updates every time new MQTT data is received.
- Gauges visually represent environmental conditions.
- Text fields display AI predictions and alerts clearly.

**Features:**

- Live data updates
- Clean UI design
- Easy interpretation
- Works over network (remote access)

## 6. Testing and Validation

Before receiving real hardware data, an **Inject Node** was used to simulate input:

```
{
 "moisture": 1200,
 "temperature": 29,
 "humidity": 70,
```

"prediction": "Soil OK",
"alert": false
}

This allowed:

- Debugging flows
- Verifying dashboard display
- Ensuring correct data extraction

7. **Outcome**

The Node-RED dashboard successfully:

- Receives real-time MQTT data
- Processes and extracts required fields
- Displays values using gauges and text widgets
- Provides clear AI-based insights for irrigation decisions

This completes the end-to-end pipeline from sensor data acquisition to intelligent visualization.

## References

[1] A. Nayyar and V. Puri, "Smart Farming: IoT Based Smart Sensors Agriculture Monitoring System," *International Journal of Scientific & Technology Research*, vol. 8, no. 8, pp. 1–8, 2019.

[2] S. K. Vishwakarma, P. Upadhyaya, B. Kumari and A. K. Mishra, "Smart Irrigation System using IoT and ESP32," *2020 3rd International Conference on Intelligent Sustainable Systems (ICISS)*, Thoothukudi, India, 2020, pp. 1-6.

[3] R. K. Kodali and S. Yerroju, "IoT based Smart Irrigation System using ESP32 and MQTT Protocol," *2021 IEEE International Conference on Intelligent Systems and Green Technology (ISGT)*, 2021.

[4] Capacitive Soil Moisture Sensor - Theory and Practice, DF Robot. [Online]. Available: https://wiki.dfrobot.com/Capacitive_Soil_Moisture_Sensor_SKU_SEN0193

[5] OpenWeatherMap API Documentation, OpenWeather Ltd. [Online]. Available: https://openweathermap.org/api

[6] A. Geron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O'Reilly Media, 2019. (Chapter on Logistic Regression)

[7] MQTT Version 5.0 Specification, OASIS Standard, 2019. [Online]. Available: https://mqtt.org/

[8] Node-RED Documentation, OpenJS Foundation. [Online]. Available: https://nodered.org/documentation/

[9] J. Hill, "Logistic Regression for Embedded Systems," *Embedded Systems Programming*, vol. 32, no. 5, pp. 45–52, 2020.

## Contribution

1. **Vithushan E.T.L. (E/20/416)**
   Hardware simulation implementation using Wokwi + Platform.io in VS code to get the data from capacitive moisture sensor and OpenWeather api.

2. **Eniyavan T. (E/20/099)**
   ML model training with custom simulated data to predict the water needs by using Logistic Regression model.

3. **Jegatheesan S. (E/19/174)**
   Node-red dashboard configuration and MQTT implementation to fetch the data from esp32 to visualize the parameters reading and show alerts.

4. **Rishanthan S. (E/20/338)**
   Integrated the lightweight Logistic Regression model inside the esp32 and published the parameters and the prediction to the Node-red via MQTT protocol.